Beyond E2E Scripts: Using LLM-Proposed Scenarios Without Letting the LLM Be the Oracle

Draft manuscript

May 2026

Abstract

Large language models can help testers explore application states that are not covered by ordinary end-to-end scripts, but they are unsafe as test oracles. They can suggest plausible scenarios, misread application semantics, and overstate weak evidence. Bayesilisk separates these roles. App-specific connectors publish observable context, available actions, and deterministic rules. Bayesilisk expands the supplied scenario space, ranks probes, and verifies only observed evidence returned by the connector. This draft reports an initial Cal.com case study at commit `180ede28f0bddf2738933a6e60a8e80f6116d7da`: three connector-supplied source facts produced six Bayesilisk probe proposals, and all six local Playwright executions reproduced a mismatch where unknown or stale `rescheduleUid` values opened booking flows with semantic status `200` although the supplied invariant expected `404`. Nearby upstream E2E tests passed, a Playwright-only connector baseline found three unknown-`rescheduleUid` failures, and a weak local model produced three invalid proposals that the verifier rejected. A browser-driving LLM-agent baseline was repeated ten times: the agent selected the malformed URL in six runs, produced invalid selections in three runs, and mismatched the deterministic oracle in three runs. A longer connector baseline also reproduced a cancelled-booking replay sequence. The remaining limitation is architectural: Bayesilisk does not yet generically synthesize bounded multi-action sequences while keeping the connector as a pure executor.

1 Problem

E2E suites are precise but narrow. LLM-driven agents are broad but unreliable: they can drive a browser, invent scenarios, and then incorrectly decide whether the behavior is acceptable. The core research claim behind Bayesilisk is that these jobs should be separated. An LLM or coding agent may help propose scenario spaces, but deterministic verifier rules must decide pass or fail.

The intended workflow is:

1. A connector extracts app-specific context: routes, actors, identifiers, available actions, nearby tests, and expected behavior.
2. Bayesilisk expands only the supplied rule space and produces probe proposals.
3. The connector executes those proposals through the real app or its test harness.
4. Bayesilisk verifies returned evidence against deterministic invariants and emits issue-ready findings.

The connector is app-specific. The Bayesilisk core is not. This boundary is the main architectural point: app teams write connectors; Bayesilisk provides the generic expansion, ranking, and verification loop.

2 Cal.com Case Study

We evaluated the current connector architecture on a local Cal.com checkout. The tested repository and revision were:

- Repository: `https://github.com/calcom/cal.com`
- Commit: `180ede28f0bddf2738933a6e60a8e80f6116d7da`
- Commit date: `2026-05-14 19:30:21 +0000`
- Commit subject: font-stack fallback fix for non-Latin script support (`#29346`)
- Bayesilisk seed: `150`

The checked-in Bayesilisk artifacts are in `examples/calcom/`. They include the connector, source context, generated proposals, execution context, JSON report, Markdown report, and issue payloads.

Artifact	Path
Connector code	`examples/calcom/bayesilisk-probes.e2e.ts`
Source context	`examples/calcom/source-context.json`
Generated proposals	`examples/calcom/generated-proposals.json`
Execution evidence	`examples/calcom/execution-context.json`
Bayesilisk report	`examples/calcom/reports/report.json`
Issue payloads	`examples/calcom/reports/issue-payloads.json`

Table 1: Checked-in Cal.com artifacts.

The connector supplied three source facts. These facts described Cal.com routes and explicit proposal rules around `rescheduleUid`. Bayesilisk generated six probe proposals: unknown and stale `rescheduleUid` values for public, private, and dynamic booking routes. The connector then executed those six proposals with Playwright against local fixtures.

Area	Mutation	Expected	Observed
Public booking page	unknown id	404	200
Public booking page	stale id	404	200
Private booking link	unknown id	404	200
Private booking link	stale id	404	200
Dynamic booking page	unknown id	404	200
Dynamic booking page	stale id	404	200

Table 2: Verified Cal.com connector observations.

The result is useful because the pass/fail verdict did not come from a model. The connector returned concrete observed facts. Bayesilisk compared `expectedStatus` and `observedStatus`, classified the six observations as `breakage.context-observed`, and produced issue payloads. An upstream issue was opened from this evidence [1].

3 Evidence Still Needed

The Cal.com result demonstrates the connector split, a real route/state mutation finding, and three baseline comparisons.

It does not yet demonstrate the full paper claim. The remaining evidence needs are generic Bayesilisk synthesis of bounded multi-action runs from connector-declared capabilities, plus broader LLM-agent baseline coverage across more models and prompts.

These baselines matter because the argument is not merely that browser automation can find a failing route. The intended claim is stronger: generic context-to-probe expansion plus deterministic verification can recover useful failures while avoiding the oracle mistakes of plain LLM agents.

Workflow	Observed scope	Failures	Notes
Nearby E2E	14 specs	0 unexpected	13 passed, 1 skipped
Playwright-only connector	11 observations	4	Includes cancelled UID replay
Bayesilisk proposal run	6 proposals	6	Unknown and stale variants
Weak local model	3 raw proposals	0 accepted	3 verifier rejections
LLM-agent browser	10 runs	3 mismatched	Stochastic oracle failures

Table 3: Cal.com baseline comparison.

4 Discussion

The current evidence supports a narrow claim: given connector-supplied source facts and rules, Bayesilisk can expand a small scenario space, execute it through an app-specific connector, and verify evidence without letting the LLM decide the outcome. This is already different from a plain Playwright script because the connector does not enumerate every final generated probe by hand; it exposes the local state and rule space, and Bayesilisk produces the concrete mutations.

The remaining work is empirical. The paper needs more app artifacts and architecture work before it can claim generality for longer workflows. The right next layer is a generic sequence proposer: connectors declare actions and state facts, while Bayesilisk generates bounded action sequences and verifies the returned evidence. The immediate target should be a concise artifact-backed paper rather than a broad methodology claim.

5 Conclusion

Bayesilisk treats LLMs as scenario proposers, not as test oracles. The Cal.com case study shows the architecture working on a real app checkout with reproducible connector evidence, baseline logs, weak-model rejection evidence, and issue-ready findings. The next version of this manuscript should add a full generic sequence-proposal layer and an LLM-agent-only browser run that actually compares additional models and prompting strategies.

References

[1] Bayesilisk contributors. Cal.com issue 29399: Booking page opens instead of 404 for missing rescheduleuid. `https://github.com/calcom/cal.diy/issues/29399`, 2026. Issue report generated from Bayesilisk evidence.